

■ Source: Sommerville SE 10th Edition + Lecture
Slides

■ Topics: 6 Major Topics + Q&A;

Table of Contents

1. Introduction to Quality Management

- 1.1 Parts of Quality Management
- 1.2 Quality Management Activities
- 1.3 QM and Software Development Diagram
- 1.4 The Feedback Loop

2. Quality Planning

- 2.1 What is a Quality Plan?
- 2.2 Humphrey's Quality Plan Structure
- 2.3 The Most Important Attribute Problem

3. Software Quality

- 3.1 Why Specification Conformance Breaks Down
- 3.2 Fitness for Purpose
- 3.3 Non-Functional Quality Attributes
- 3.4 The 15 Quality Attributes Explained
- 3.5 Process Quality vs. Product Quality

4. Software Standards

- 4.1 Types of Standards
- 4.2 Product Standards vs. Process Standards
- 4.3 Why Standards Are Important
- 4.4 Problems with Standards

5. Reviews and Inspections

- 5.1 What Are Reviews?
- 5.2 Types of Reviews
- 5.3 Three Phases of the Review Process
- 5.4 Distributed Reviews
- 5.5 Program Inspections
- 5.6 Inspection Checklists
- 5.7 Reviews vs. Inspections Comparison

6. Software Measurement

- 6.1 What is Software Measurement?
- 6.2 Types of Process Metrics
- 6.3 Internal vs. External Attributes
- 6.4 Static Software Product Metrics
- 6.5 Object-Oriented Metrics

7. Student Q&A; — Clarifications

8. Full Chapter Revision Summary

Section 1: Introduction to Quality Management

Software Quality Management (QM) is concerned with ensuring that developed software systems are **fit for purpose** — they meet user needs, perform efficiently and reliably, and are delivered on time and within budget.

1.1 The Four Parts of Quality Management

Quality Management consists of four interconnected activities that form a **continuous cycle** — each feeding into the next.

Part	Core Question	Description
Quality Planning	What quality do we need?	Identify what could be improved; define quality standards and targets before development begins.
Quality Assurance (QA)	Are we doing it right?	Use quality processes and standard procedures to develop the product. Involves Verification & Validation.
Quality Control (QC)	Did we do it right?	Test the products and identify defects to improve product quality.
Quality Improvement	How do we do better?	Improve product quality continuously in response to emerging requirements and feedback.

Think of these as a **cycle, not isolated steps**:

Quality Planning → Quality Assurance → Quality Control → Quality Improvement →
(feeds back to Planning)

1.2 Quality Management Activities

The QM process continuously runs alongside software development and performs the following activities:

- Ensures the required level of quality is achieved in the software product.
- Continuously checks the software development process — not just at the end.
- Checks that project deliverables are aligned with organisational standards and goals.
- Maintains process documentation so that no task is forgotten and no team makes incorrect assumptions.
- In large companies, manages the release testing process before the product is shipped to customers.

Why Must the QM Team Be Independent?

This is one of the most frequently examined principles. The QM team must be **independent from the development team** for the following reasons:

If QM is Part of Dev Team	If QM is Independent
May hide defects to meet deadlines	Reports issues honestly and objectively
Influenced by schedule/budget pressure	No conflict of interest
Quality assessments are biased	Objective judgment is maintained
Quality suffers for short-term goals	Organisational quality goals are protected

Analogy: A football referee cannot be a player on either team — their independence is what makes their judgment trustworthy.

Note from Sommerville: In small companies, this independence is practically impossible — developers often carry both development and quality responsibilities. The book acknowledges this real-world constraint.

1.3 QM and Software Development — The Diagram Explained

The diagram from the lecture slides shows **two parallel timelines** running side by side throughout the project:

Software Dev Process → D1 → D2 → D3 → D4 → D5

↓ ↓ ↓ ↓ ↓

QM Process → checks at every deliverable

D1–D5 represent **Deliverables** — outputs produced at each stage. Every time development produces a deliverable, the QM process receives and checks it. The QM process produces three key outputs:

QM Output	What It Means
Standards & Procedures	Rules defined at the start — how work should be done.
Quality Plan	The roadmap — what quality targets to hit and when.
Quality Review Reports	Ongoing feedback — findings after checking each deliverable.

1.4 The Feedback Loop

After a Quality Review Report is generated, the development team performs **two types of action**:

Action Type	Purpose
Corrective Action	Fix the defects found in the current deliverable.
Preventive Action	Change process/habits so the same mistake does not appear in future deliverables.

Analogy: Like a teacher marking your assignment — you correct the mistakes on that assignment (corrective) and learn not to repeat them in the next one (preventive).

■ IMPORTANT EXAM POINTS

- ◆ QM team must be independent — reports above project manager level to avoid schedule/budget bias.
- ◆ QM runs CONTINUOUSLY alongside development — not just at the end.
- ◆ The four parts form a cycle: Planning → Assurance → Control → Improvement.
- ◆ Corrective action = fix current defects. Preventive action = avoid future repetition.

■ QUICK REVISION SUMMARY

- QM ensures fitness for purpose, not just spec conformance.
- Operates at organisational level (company-wide) and project level (specific project).
- QM team checks deliverables, process docs, and manages release testing.
- Independence of QM = objectivity — no schedule/budget influence.
- Feedback loop: Review Report → Dev team fixes + learns → fewer defects in next deliverable.

Section 2: Quality Planning

Quality Planning answers the fundamental question: '**What does quality actually mean for THIS specific project?**' This sounds obvious but is one of the hardest problems in Software Engineering because quality means different things for different systems.

2.1 What is a Quality Plan?

Definition (Sommerville): The quality plan sets out the desired software qualities and describes how these qualities are to be assessed. It defines what '*high-quality*' software actually means for a particular system.

From the lecture slides, a quality plan:

- Lists the **desired product qualities**.
- Describes **how they should be assessed**.
- Identifies the **most significant quality attributes** (e.g., reliability or efficiency).
- Should be **short and succinct** — if too long, people will not read it, defeating its purpose.

Analogy: A Quality Plan is like an architect's blueprint — written before construction, it defines what 'built well' looks like for that specific building.

2.2 Humphrey's Quality Plan Structure

Sommerville references Humphrey's classic 5-part structure for quality plans:

Section	Contents
1. Product Introduction	Description of the product, intended market, and quality expectations.
2. Product Plans	Critical release dates, responsibilities, distribution plans, and servicing plans.
3. Process Descriptions	Development and service processes and standards to be used.
4. Quality Goals	Quality goals for the product, most critical attributes, and justification.
5. Risks & Risk Management	Key risks that might affect product quality and actions to address them.

2.3 The Most Important Attribute Problem

It is **impossible** to optimise any system for all quality attributes simultaneously. They trade off against each other. The quality plan must therefore explicitly **prioritise** the most important attributes for that specific product.

System Type	Priority Attributes	Reason
Pacemaker Software	Safety, Reliability	Failure costs human lives.
Banking System	Security, Reliability	Financial data must be protected.
Gaming Application	Performance, Usability	User experience is the product.
Air Traffic Control	Safety, Reliability	Catastrophic consequences of failure.
Fitness Mobile App	Usability, Performance	Users delete slow or confusing apps.

Classic attribute trade-offs to remember:

- Improving **Security** → may reduce **Performance** (encryption overhead).
- Improving **Reusability** → may increase **Complexity** (generic code is slower).
- Improving **Portability** → may reduce **Performance** (abstraction layers add overhead).
- Rushing for **Efficiency** → may reduce **Maintainability**.

■ IMPORTANT EXAM POINTS

- ◆ Quality plans must be SHORT — a long plan nobody reads provides zero quality benefit.
- ◆ Humphrey's 5-part structure: Product Intro → Product Plans → Process Descriptions → Quality Goals → Risks.
- ◆ No system can be optimised for ALL quality attributes — the plan must PRIORITISE.
- ◆ Quality Plan is written BEFORE development begins — it is the foundation for all QM activities.

■ QUICK REVISION SUMMARY

- Quality Plan = blueprint defining what 'quality' means for a specific project.
- 5-part structure: Introduction, Plans, Process, Goals, Risks.
- Must prioritise quality attributes — you cannot have everything.
- Must be short and readable or it defeats its own purpose.

- Feeds directly into QA (standards), QC (assessment methods), and QI (improvement targets).

Section 3: Software Quality

3.1 Why Specification Conformance Breaks Down in Software

Quality in manufacturing is simple: a product either matches its precise specification (within tolerance) or it does not. **Software quality cannot be measured this way** for three fundamental reasons:

Reason	Explanation	Real Example
Specs are incomplete & inconsistent	Requirements are written by humans — ambiguous, contradictory, or missing edge cases.	"The system shall be fast" — how fast exactly?
Stakeholder conflicts	Customer wants reliability. Developer wants maintainability. The spec is always a compromise.	A hospital system spec satisfies admin but frustrates nurses.
Some qualities cannot be measured directly	How do you measure maintainability before the software ages?	You only know code is unmaintainable after 2 years of struggling with it.

3.2 Fitness for Purpose

Key Definition: Rather than asking 'Does this software match the written spec?', the correct question is: **'Does this software actually serve its users well in the real world?'**

The Sommerville textbook calls this **Fitness for Purpose** — the software must genuinely meet its users' needs in practice, not just on paper.

Six checks used to assess fitness for purpose:

Check	Why It Matters
Has the software been properly tested?	Testing proves requirements are implemented and working.
Is performance acceptable for normal use?	Slow software frustrates users even if functionally correct.
Is the software usable?	Unusable software fails its purpose regardless of correctness.

Is the software well-structured and understandable?	Poor structure makes future maintenance a nightmare.
Have programming standards been followed?	Consistency reduces bugs and helps new developers.
Have documentation standards been followed?	Future teams cannot maintain what they cannot understand.

3.3 Functional vs. Non-Functional Quality

Important Clarification from the tutoring session: Functional requirements are the **foundation** — without them, software has no purpose at all. The key insight is about how users *react* to failures in each layer:

Type of Failure	User Reaction	Fatal to Product?
Minor functional bug (e.g., sort button broken)	User finds a workaround and continues.	Not always fatal
Non-functional failure (e.g., 45-second load time, frequent crashes)	User switches to competitor.	Usually fatal
Major functional failure (core features broken)	Software is completely unusable.	Always fatal

Refined statement: Functional correctness is necessary but not sufficient. Once basic functionality exists, real-world quality is dominated by non-functional attributes — because users can tolerate minor functional gaps but cannot work around poor performance, unreliability, or poor usability.

Analogy: If your GPS app gives slightly wrong directions occasionally, you double-check manually. But if it freezes every 2 minutes — you delete it. The non-functional failure is fatal; the minor functional one is tolerable.

3.4 The 15 Quality Attributes — Explained

The lecture slides identify 15 key software quality attributes. It is **impossible** for any system to be optimised for all of them simultaneously. Below is a complete explanation of each:

#	Attribute	Definition	Example Concern
1	Safety	The system must not cause harm to users or the environment in the event of a failure.	A pacemaker must not deliver incorrect electrical pulses.

2	Security	The system must protect itself and its data from unauthorised access or malicious attacks.	A banking app must prevent unauthorised transactions.
3	Reliability	The system must perform its required functions correctly and consistently over time without failure.	An ATM must dispense the correct amount every single time.
4	Resilience	The system must continue operating — possibly in a degraded mode — when parts of it fail or are attacked.	A web server should serve cached pages even if the database is temporarily down.
5	Robustness	The system must handle unexpected or invalid inputs without crashing or producing incorrect results.	A form should not crash if a user enters letters in a numeric field.
6	Efficiency	The system must make optimal use of computational resources — time, memory, and energy.	A mobile app must not drain the phone battery within an hour.
7	Usability	The system must be easy for its intended users to learn and use effectively.	A hospital record system must allow a nurse to find a patient record within seconds.
8	Learnability	New users must be able to learn how to use the system quickly without extensive training.	A checkout system must be learnable by a new cashier within one shift.
9	Understandability	The system's purpose, structure, and code must be clear and easy to comprehend by developers.	Another developer should understand a module's purpose within minutes of reading it.
10	Testability	The system must be structured such that it can be easily and thoroughly tested.	Each module should have clear inputs and outputs that can be verified independently.
11	Adaptability	The system must be able to be adapted for use in different environments or with different data without major changes.	An ERP system deployed in both Pakistan and the UK must handle different tax rules.
12	Modularity	The system must be built from separate, self-contained components with well-defined interfaces.	A payment module can be updated independently without affecting the inventory module.
13	Reusability	Components of the system must be usable in other systems or contexts without significant modification.	An authentication module written for one product can be reused in another product.

1 4	Portability	The system must be transferable to different hardware or software environments with minimal effort.	A web application must run on Chrome, Firefox, and Safari without separate versions.
1 5	Complexity	The internal structural complexity of the system must be kept low to reduce defect risk and maintenance difficulty.	A class with 50 methods and deep nesting will be hard to test and maintain.

3.5 Process Quality vs. Product Quality

Traditional assumption (from manufacturing): **Good process** → **Good product**. The process-based quality model works as follows:

```
Define process → Develop product → Assess quality
```

```
↑ |
```

```
■■■ if quality not OK, improve ■■■■■■
```

However, **software is designed, not manufactured**. Key differences:

- Individual creativity and skill matter enormously — a brilliant developer on a poor process can still produce great software.
- External pressures (tight deadlines, novel problems) affect quality independently of process.
- Process standardisation can sometimes **stifle creativity**, leading to poorer software.

Sommerville therefore argues for developing a '**quality culture**' — every team member is personally committed to quality, not just following checklists.

■ IMPORTANT EXAM POINTS

- ◆ Fitness for purpose > specification conformance — the key definition of software quality.
- ◆ Software specs are incomplete, ambiguous, and reflect stakeholder compromises.
- ◆ Non-functional failures are often MORE fatal to products than minor functional bugs.
- ◆ Know all 15 quality attributes and understand the trade-offs between them.
- ◆ Process quality helps but does not guarantee product quality in software — creativity matters.

■ QUICK REVISION SUMMARY

- Spec conformance breaks down because: specs are incomplete, stakeholders conflict, some qualities unmeasurable.

- Fitness for purpose = does the software genuinely serve its users in real use?
- Functional requirements = baseline (must-have). Non-functional attributes = determine real quality.
- 15 quality attributes — impossible to optimise all; quality plan must prioritise.
- Quality culture — everyone committed to quality, not just process-followers.

Section 4: Software Standards

Standards answer: **'How does a software organisation ensure that quality is consistently achieved — not just on one lucky project, but on EVERY project?'**

Definition: Standards define how the software process should be put into practice. They play an important role in quality management.

Analogy: McDonald's has a standard that a burger must be cooked for exactly X seconds at Y temperature in every branch worldwide. That consistency is achieved through standards, not luck. Software standards do the same for development teams.

4.1 Types of Standards — Two Classifications

Classification 1: By Scope

Type	Examples
External Standards	International (ISO), National (IEEE) standards — apply across the industry.
Internal Standards	Organisational standards, project-specific standards set by stakeholders.

Classification 2: By Application (Most Important for Exams)

	Product Standards	Process Standards
Applied to	The software product being built	The process followed to build it
Focus	What the OUTPUT looks like	How the WORK is done
Question answered	'Does our code/doc look right?'	'Did we follow the right steps?'
Examples	Document structure, coding style, comment headers	Design review procedure, version release process, change control

4.2 Product Standards — In Depth

1. Document Standards

Define the structure of documents like SRS (Software Requirements Specification) or SDS (Software Design Specification).

Example: Every SRS must contain: Introduction → Scope → Functional Requirements → Non-Functional Requirements → Glossary.

2. Code Documentation Standards

Standard comment headers for every class or function. Example:

```
// Author: XYZ | Date: 08-May-2026
// Purpose: Calculates monthly salary
// Parameters: basePay, allowances | Returns: totalSalary
```

3. Coding Standards

Rules for how a programming language must be used:

- Use proper naming conventions (*calculateSalary* not *cs*).
- Class members must be accessed privately.
- Never leave a catch block empty.
- Use underscores in lengthy numeric literals: 1_000_000 not 1000000.

4.3 Process Standards — In Depth

Process standards define **how work should be done**, not what outputs look like. They include the tools used to support the process, such as the **Fishbone Diagram** (Ishikawa diagram) — used to identify the causes and effects of a problem on a product.

Process Standard	What It Defines
Design review conduct	How a design review meeting must be structured and run.
Submission of new code	Steps to follow when submitting code to the build system.
Version release process	How software versions are approved and released to customers.
Change control process	How requirement changes are formally requested and approved.
Project plan approval	Who must sign off on a project plan before work begins.
Test recording process	How test results must be documented and stored.

4.4 Product and Process Standards — Full Comparison Table

Product Standards (Forms, Formats, Structures)	Process Standards (Conduct, Activity, Approval)
--	---

Design review form	Design review conduct
Requirements document structure	Submission of new code for system building
Method header format	Version release process
Java programming style	Project plan approval process
Project plan format	Change control process
Change request form	Test recording process

Pattern to remember: Product standards = *forms, formats, structures* (the artifact). Process standards = *conduct, process, approval* (the activity).

4.5 Why Standards Are Important — 3 Reasons

Reason	Explanation
1. Capture organisational wisdom	Standards are built from hard-won experience — past mistakes and lessons learned. They prevent teams from repeating old errors on new projects.
2. Define what quality means	Without standards, quality is just an opinion. Standards provide an objective baseline — if the code follows the standard, it meets a defined level of quality.
3. Ensure continuity	When a developer leaves and a new one joins, standards ensure the new person can understand and continue the work without starting from scratch.

4.6 Problems with Standards

Problem	Explanation	Sommerville's Solution
Get outdated	Technology evolves fast; a standard written 5 years ago may no longer reflect best practices.	Review and update standards regularly as technology changes.
Too much documentation overhead	Engineers spend more time filling forms than writing code.	Provide tool support to automate standard enforcement (e.g., code formatters).

Too rigid	Different software types need different processes — one standard cannot fit all.	Involve engineers in selecting standards — they will commit to ones they helped choose.
-----------	--	---

■ IMPORTANT EXAM POINTS

- ◆ Two types of standards: Product (what output looks like) and Process (how work is done).
- ◆ Three reasons standards matter: capture wisdom, define quality, ensure continuity.
- ◆ Three problems: get outdated, too much overhead, too rigid.
- ◆ Sommerville's three solutions: involve engineers, update regularly, provide tool support.
- ◆ Pattern: Product standards = forms/formats. Process standards = conduct/procedures.

■ QUICK REVISION SUMMARY

- Standards = rules ensuring consistent quality across all projects and all developers.
- External (ISO/IEEE) vs. Internal (org/project level).
- Product standards apply to OUTPUT. Process standards apply to HOW WORK IS DONE.
- 3 reasons: capture wisdom, define quality, ensure continuity.
- 3 problems + solutions: outdated→update regularly; overhead→tool support; rigid→involve engineers.

Section 5: Reviews and Inspections

Reviews and Inspections answer: 'How do we check whether quality standards are being met during development — before it is too late?'

5.1 What Are Reviews and Inspections?

Definition (Sommerville): A group examines a part of the process or system and its documentation to find potential problems.

They are classified as **Quality Assurance activities** that run **side by side with Verification and Validation** throughout the SDLC. They check:

- The software itself.
- The documentation.
- The development process.
- Non-conformance with any standard.
- Defects in deliverables.

5.2 Why Reviews Instead of Just Testing?

The golden rule: **A bug found early costs dramatically less to fix than one found late.**

A requirements error found in a review costs almost nothing to correct. The same error found after deployment may require redesigning entire modules and re-testing everything.

Analogy: Proofreading an essay before printing 1,000 copies is far cheaper than reprinting after spotting the error.

Additionally, once a system is finalised, all modules are interconnected. Fixing one bug at this stage can **break other modules** that relied on the buggy behaviour — a 'ripple effect'. Reviews catch defects early, before dependencies are built on top of flawed foundations.

5.3 What Do Reviews Assess?

Important: Reviews are NOT limited to documents only. They assess **everything produced in the SDLC**:

Item Reviewed	Example Check
Requirements specification	Is the SRS complete and unambiguous?

Design documents	Does the design match the requirements?
Code	Does the code follow coding standards? Is it consistent with the design?
Process models	Are the right development processes being followed?
Test plans	Do tests cover all requirements?
Process standards	Are team members following defined procedures?
User manuals	Is documentation clear and accurate?

Reviews check for: **Consistency** (does everything align?), **Completeness** (is anything missing?), **Conformance to standards** (are rules being followed?), and **Defects** (what is wrong?).

5.4 Types of Reviews

Type	Purpose	Key Question
Quality Reviews	Check the software product quality.	'Is what we built GOOD?'
Progress Reviews	Check whether the project is on schedule.	'Are we building it ON TIME?'

5.5 Three Phases of the Review Process

This is a **very exam-likely topic**.

Phase 1 — Pre-Review Activities (Before the Meeting)

- Schedule the review time and location.
- Distribute documents to all reviewers in advance.
- Reviewers read and prepare comments **before** arriving.

Why this matters: You cannot walk into a review meeting cold. Reviewers must have studied the document beforehand — otherwise the meeting wastes everyone's time.

Phase 2 — The Review Meeting (During)

- The review team examines the document/code against organisational and project-level standards.
- **Must be short — maximum 2 hours.**
- All comments and defects are noted and recorded.
- The author of the document is present but listens — does not defend.

Why max 2 hours? After 2 hours, attention drops, people get tired, and review quality collapses. It is better to schedule a second session than to drag one out.

Phase 3 — Post-Review Activities (After)

- Address all problems raised during the meeting.
- Actions include: fixing bugs, refactoring code, rewriting documents.
- Changes are verified to ensure issues were properly resolved.

5.6 Distributed Reviews — Modern Reality

Traditionally, reviews are conducted face-to-face. However, project teams are now often **distributed across cities or even continents**, making face-to-face meetings impractical.

Solution: Remote reviewing using **shared documents** where each review team member annotates their comments digitally — everyone can see and respond to each other's notes asynchronously.

5.7 Program Inspections

Definition: Team members collaborate to identify **anomalies and defects in code prior to execution** — meaning without running the code at all.

What 'before code execution' means: Inspectors read the code like reading a book — with their eyes, not by running it on a computer. No compiler, no terminal, no test cases. Humans read code line-by-line and spot problems mentally.

Inspection vs. Testing — Concrete Example

```
int[] arr = new int[5];  
arr[5] = 10; // Bug: index out of bounds!
```

Via Inspection: A reviewer reads this line and says: 'Array size is 5, valid indexes are 0–4, but you're accessing index 5. That is a bug.' — caught without running anything.

Via Testing: You run the program → it crashes with `ArrayIndexOutOfBoundsException` → now you debug.

Aspect	Detail
When performed	Before the code is executed or tested.
What is examined	Code, but also requirements, design, and test data.
How it works	Using a structured checklist of common error types.
Effectiveness	Up to 60% of defects can be removed through inspections alone.

■ The 60% statistic is a famous exam fact — always remember it.

5.8 Inspection Checklists

Inspections use pre-built checklists of common error types. **Checklists are programming-language dependent** — a Java checklist differs from a C checklist because languages behave differently.

Fault Class	Example Inspection Checks
Data Faults	Are all variables initialised before use? Is there buffer overflow risk? Are all constants named?
Control Faults	Is every loop guaranteed to terminate? Are all cases in a switch/case handled? Are compound statements correctly bracketed?
Input/Output Faults	Are all input variables actually used? Are all output variables assigned before display? Can unexpected input corrupt the system?
Interface Faults	Do function calls have the correct number and types of parameters? Are parameters in the right order?
Storage Management Faults	Is dynamically allocated memory properly freed? Has space been correctly allocated?
Exception Management Faults	Have ALL possible error conditions been accounted for?

5.9 Team Composition — Reviews vs. Inspections

	Reviews	Inspections
Team size	Larger, broader group	Smaller, focused technical group
Who leads	QM team or project manager	A trained Moderator
Participants	Developers, QM, managers	Developers + Moderator + Scribe
Author's role	Presents the work, listens	Clarifies when asked — does not defend
Focus	Documents, process, code broadly	Code specifically, line by line
Approach	Discussion and judgment	Structured checklist-driven

Key rule for BOTH: The author must not become defensive. The goal is finding defects, not judging the person.

Analogy: When a doctor reviews an X-ray with colleagues, the radiologist listens to what others see rather than arguing. The goal is the patient's health, not the radiologist's ego.

5.10 Reviews vs. Inspections — Final Comparison

Dimension	Reviews	Inspections
Scope	Documents + Code + Process — broadly	Code specifically — very deeply
Approach	General discussion, judgment-based	Structured, checklist-driven, line-by-line
Depth	Broad and high-level	Narrow and deep
Goal	Overall quality — consistency, completeness, standards	Find specific defects and anomalies in code
Team	Broader mixed group	Smaller technical group with defined roles
Output	Quality Review Report	List of defects/anomalies

Analogy: Review = General doctor's health check (broad). Inspection = Cardiologist's detailed heart examination (deep and specific).

■ IMPORTANT EXAM POINTS

- ◆ Inspections can remove up to 60% of defects — most powerful QA tool.
- ◆ 'Before code execution' means humans read code with eyes only — no running, no compiler.
- ◆ Review meeting must be MAX 2 HOURS — quality drops after that.
- ◆ Three phases: Pre-review (distribute) → Meeting (examine, record) → Post-review (fix).
- ◆ Reviews are NOT limited to documents — they also cover code.
- ◆ Reviews = broad/discussion-based. Inspections = deep/checklist-driven.
- ◆ Checklists are LANGUAGE-DEPENDENT — Java checklist differs from C checklist.
- ◆ Author does NOT defend in either reviews or inspections — listens and clarifies only.

■ QUICK REVISION SUMMARY

- Reviews and Inspections = QA activities running throughout SDLC, not just at the end.
- Early defect detection → dramatically cheaper to fix than post-deployment bugs.
- Reviews cover: specs, design, code, process models, test plans, standards, user manuals.
- 3 phases: Pre-review → Meeting (max 2hrs) → Post-review.
- Distributed teams use shared annotated documents for remote reviewing.
- Inspections: before execution, checklist-driven, removes 60% of defects.
- Reviews = broad team. Inspections = smaller technical team with Moderator + Scribe.

Section 6: Software Measurement

Core Question: How do we put **numbers** on software quality — so we can measure, compare, and improve it objectively?

6.1 What is Software Measurement?

Definition (Sommerville): Software measurement is the process of deriving a numeric value for an attribute of a software product or process. It allows objective comparison between techniques and processes.

Without measurement: *'I think this code is good.'* (opinion)

With measurement: *'This module has 2.3 defects per 100 lines of code.'* (fact)

Analogy: A doctor who never measures anything says 'you seem kind of healthy.' A doctor who measures says 'your blood pressure is 140/90 — that is high.' The second can track progress and make evidence-based decisions.

Reality Check: Most organisations still do not make systematic use of software measurement, despite its benefits. There are few established standards in this area.

6.2 Types of Process Metrics

Metric Type	What It Measures	Examples
Time	How long a process takes.	Calendar time to complete a module; person-days spent on design.
Resources	What was consumed to complete it.	Total effort in person-days; travel costs; compute resources.
Occurrences	How often something happens.	Number of defects found in inspection; number of requirement changes; number of bug reports; average LOC modified per requirement change.

6.3 Internal vs. External Quality Attributes

	Internal Attributes	External Attributes
What they are	Properties of the code/product itself	How the product behaves in real use

Measured by	Static analysis — reading/analysing code	Dynamic observation — using the product
Examples	Lines of code, complexity, fan-in/fan-out	Reliability, usability, performance
When measured	During development	During/after deployment

Key insight: Internal attributes *influence* external attributes. High code complexity (internal) leads to more bugs and lower reliability (external).

6.4 Static Software Product Metrics

These are measurements taken **without running the code** — purely by analysing it statically. There are six key metrics:

Metric	What It Measures	Problem If High/Low
Fan-in	Number of functions that CALL function X.	High fan-in → X is tightly coupled. Changing X breaks many other functions.
Fan-out	Number of functions that X CALLS.	High fan-out → X is very complex. Coordination of many components is difficult.
Length of Code (LOC)	Size of the program in lines of code.	High LOC → more complex, more error-prone. One of the most reliable defect predictors.
Cyclomatic Complexity	Number of independent execution paths through the code (branches, loops, conditions).	High complexity → harder to understand, test, and maintain. More likely to contain hidden bugs.
Length of Identifiers	Average length of variable, method, and class names.	Short identifiers → less meaningful, less understandable (e.g., 'x' vs 'monthlyRevenue').
Depth of Conditional Nesting	How deeply if-statements are nested inside each other.	Deep nesting → hard to read, hard to test, error-prone.
Fog Index	Average length of words and sentences in documents.	High Fog Index → harder to understand documentation.

6.5 Object-Oriented Metrics Suite

OO Metric	Full Name	What It Measures	Problem If High
WMC	Weighted Methods per Class	Number of methods × complexity of each method.	Class is too complex, hard to understand, and difficult to reuse as a superclass.
DIT	Depth of Inheritance Tree	Number of levels deep the inheritance hierarchy goes.	Must understand all parent classes to understand the child class. Very complex.
NOC	Number of Children	Number of immediate subclasses a class has.	Base class needs rigorous validation — many subclasses depend on it.

DIT Analogy: Animal → Mammal → Primate → Ape → Chimpanzee (DIT = 5). To understand Chimpanzee, you must understand all 4 parent classes. Very deep = very complex.

6.6 The Measurement → Improvement Cycle

Measure process/product → Identify weak areas objectively

→ Make targeted improvements → Measure again → Continuous improvement ■

This connects directly back to **Quality Improvement** — the 4th part of QM. Measurement is what makes improvement **evidence-based** rather than guesswork.

■ IMPORTANT EXAM POINTS

- ◆ Cyclomatic complexity → testability risk (many untested paths) + reliability risk in production.
- ◆ High cyclomatic complexity passing tests does NOT mean quality software — hidden paths untested.
- ◆ Fan-in high = tightly coupled. Fan-out high = high complexity.
- ◆ LOC is one of the most reliable predictors of defect-proneness.
- ◆ OO metrics: WMC (class complexity), DIT (inheritance depth), NOC (number of children).
- ◆ Internal attributes influence external attributes — always explain this connection.

■ QUICK REVISION SUMMARY

- Software measurement = deriving numeric values for product/process attributes.
- Process metrics: time, resources, occurrences.
- Internal (code-level) → influences → External (real-world behaviour).
- Static metrics (no code execution): Fan-in/out, LOC, Cyclomatic Complexity, Identifier Length, Nesting, Fog Index.
- OO metrics: WMC, DIT, NOC.
- Most organisations don't yet use measurement systematically — but it enables evidence-based improvement.

Section 7: Student Q&A; — Key Clarifications

This section preserves the important conceptual questions raised during the tutoring session, along with their full explanations.

Q1: If software doesn't perform functional requirements, it becomes useless — how can you say non-functional attributes dominate quality?

■ STUDENT QUESTION

If the software is not performing functional requirements, it becomes of no use. So how can you say non-functional characteristics dominate quality?

Answer: You are absolutely right to challenge this. The accurate statement is a two-layer model:

Layer 1 — Functional Requirements = MUST HAVE (baseline). A fitness app that does not track steps is useless — full stop. Functional requirements are the minimum entry ticket.

Layer 2 — Non-Functional Attributes = Determine Real Quality. Once basic functionality exists, real-world quality is dominated by non-functional attributes.

The difference is how users *react* to partial failures:

- **Minor functional bug** (sort button broken) → User finds a workaround → Not always fatal.
- **Non-functional failure** (45-second load time, crashes) → User deletes app → Usually fatal.
- **Major functional failure** (core features broken) → Software is completely useless → Always fatal.

Exam-ready answer: Functional correctness is necessary but not sufficient for quality. Once a system satisfies its basic functional requirements, its perceived quality is dominated by non-functional attributes — because users can tolerate minor functional gaps but cannot work around a system that is too slow, unreliable, or unusable.

Q2: What does 'before code execution' mean in Program Inspections?

■ STUDENT QUESTION

In inspection, what do you mean by 'before code execution'? Are we not running the software even once?

Answer: Correct — in a Program Inspection, the team reads the code **like reading a book** — with their eyes only. No compiler, no terminal, no test cases. Humans read code line by line and spot problems mentally.

Example:

```
int[] arr = new int[5];  
arr[5] = 10; // Bug: index out of bounds!
```

An inspector reading this identifies the bug immediately: array size 5 means valid indexes 0–4, but index 5 is being accessed. This is caught without running anything.

This is valuable because some bugs — security vulnerabilities triggered by rare conditions, memory leaks appearing after 10,000 users — would never be caught in normal testing.

Q3: Is there a separate team for Reviews and a separate team for Inspections?

■ STUDENT QUESTION

Do we have a separate team for reviews and a different team for inspections?

Answer: No — but the team *composition* differs significantly:

	Reviews	Inspections
Team size	Larger, broader group	Smaller, focused technical group
Who leads	QM team or project manager	A trained Moderator
Participants	Developers, QM team, managers	Developers + Moderator + Scribe
Author's role	Presents work, listens	Clarifies when asked, does not defend

Key rule for both: The author must not become defensive. The goal is finding defects, not judging the person.

Q4: Do Reviews only cover documents, or also code?

■ STUDENT QUESTION

In reviews, do we only review documents? We don't look at code?

Answer: No — reviews are NOT limited to documents. The lecture slide explicitly states that reviews assess: 'Documents developed in SDLC like specifications, design, **code**, process models, test plans, process standards and user manuals.'

Code is explicitly listed. The real difference between reviews and inspections is **not what they look at** — it is **how they look at it**:

Dimension	Reviews	Inspections
Scope	Documents + Code + Process broadly	Code specifically, very deeply
Approach	General discussion, judgment-based	Structured, checklist-driven, line-by-line
Depth	Broad and high-level	Narrow and deep

Section 8: Full Chapter Revision Summary

A complete at-a-glance summary of all six topics for exam preparation.

#	Topic	Core Idea	Key Exam Points
1	QM Overview	Independent QM team ensures objective, unbiased quality checking throughout development.	Independence → objectivity. Two levels: org + project. QM runs continuously.
2	Quality Planning	Define what quality means for THIS project before building begins.	Humphrey's 5-part plan. Must be short. Cannot optimise all attributes.
3	Software Quality	Fitness for purpose > specification conformance. Non-functional attributes dominate.	3 reasons specs fail. 15 quality attributes. Quality culture. Process ≠ guarantee.
4	Software Standards	Consistent rules for product and process — capture wisdom, define quality, ensure continuity.	Product vs. Process standards. 3 reasons. 3 problems + 3 solutions.
5	Reviews & Inspections	Catch defects early through structured examination throughout the SDLC.	60% defect removal. Max 2hr meeting. 3 phases. Reviews ≠ docs only.
6	Software Measurement	Quantify quality objectively to enable evidence-based improvement.	Fan-in/out, LOC, Cyclomatic Complexity. WMC, DIT, NOC. Internal→External link.

Master Exam Points — All Topics

■ IMPORTANT EXAM POINTS

- ❖ QM team must be INDEPENDENT — reports above project manager level.
- ❖ QM runs CONTINUOUSLY — parallel to development, not just at the end.
- ❖ Quality Plan = short, succinct — define quality BEFORE development starts.
- ❖ Fitness for purpose > specification conformance — the core definition of software quality.
- ❖ Functional requirements = baseline. Non-functional attributes = determine real quality.

- ◆ 15 quality attributes — impossible to optimise all simultaneously.
- ◆ Product standards = forms/formats/structures. Process standards = conduct/procedures/approvals.
- ◆ 3 reasons for standards: capture wisdom, define quality, ensure continuity.
- ◆ 3 problems with standards + solutions: outdated→update; overhead→tools; rigid→involve engineers.
- ◆ Reviews cover: specs, design, CODE, process models, test plans, standards, user manuals.
- ◆ Inspections = before code execution, checklist-driven, removes 60% of defects.
- ◆ Review meeting = MAX 2 HOURS. 3 phases: pre-review, meeting, post-review.
- ◆ Reviews = broad/discussion. Inspections = deep/checklist. Both: author does NOT defend.
- ◆ Checklists are programming-language dependent.
- ◆ Fan-in high = tightly coupled. Fan-out high = high complexity.
- ◆ LOC = most reliable predictor of defect-proneness.
- ◆ High cyclomatic complexity passing tests does NOT mean quality — hidden paths untested.
- ◆ OO metrics: WMC (method complexity), DIT (inheritance depth), NOC (number of children).
- ◆ Measurement enables evidence-based improvement — connects to Quality Improvement cycle.

These notes were generated from a complete interactive tutoring session combining Sommerville's Software Engineering (10th Edition) with lecture slides on Chapter 24: Quality Management. All concepts, analogies, tables, and Q&A; interactions are preserved for maximum exam preparation value.